INGENIERÍA DE SOFTWARE II · ENUNCIADO DEL TALLER

Patrones de Arquitectura

Cinco patrones, cinco retos sobre el mismo sistema de préstamos de la biblioteca.



Organización del sistema · Presentación · Flujo y comunicación

Instrucciones

Cada uno de los cinco patrones viene con un código que ya funciona y un reto corto. Todo el taller usa el mismo ejemplo: los préstamos de la Biblioteca UN.

- Grupos de máximo **3 personas**. Necesitan Python 3.10 o superior; no hay que instalar nada más.
- Descarguen la carpeta `codigo-base`. Hay un archivo por patrón (`1_capas.py`, `2_hexagonal.py`…).
- Resuelvan cada reto en su archivo. Al ejecutarlo (`python 1_capas.py`) se debe ver que funciona.
- No borren ni reescriban el código que ya está: agreguen lo que pide el reto. Solo cambien una línea existente cuando el reto lo diga.

Entrega

Los 5 archivos `.py` resueltos y un `README.md` con los nombres del grupo. Cada reto vale 1 punto: el archivo corre sin errores y hace lo que pide el reto.

Contenido

Organización del sistema **2**
1.1 Capas 2
1.2 Hexagonal (puertos y adaptadores) 4

Presentación **6**
2.1 Modelo–Vista–Controlador (MVC) 6

Flujo y comunicación **8**
3.1 Orientado a eventos 8
3.2 Tuberías y filtros 10

Organización del sistema

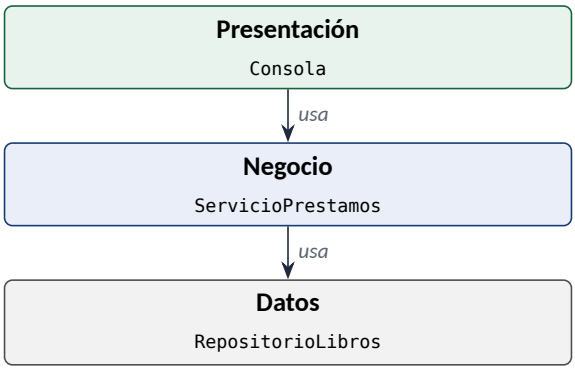
Capas y hexagonal.

1.1 Capas

Entregable: `1_capas.py`

Cada capa solo llama a la de abajo

El sistema está dividido en tres capas: **presentación** (`Consola`), **negocio** (`ServicioPrestamos`) y **datos** (`RepositorioLibros`). La consola nunca toca los datos directamente: siempre pasa por el servicio.



Código base — `1_capas.py`

```python
"""Arquitectura en capas: préstamos de la Biblioteca UN."""


# ================== Capa de datos ==================
class RepositorioLibros:
    def __init__(self):
        self.libros = {"L1": "Clean Architecture", "L2": "Refactoring"}
        self.prestados = set()


# ================= Capa de negocio =================
class ServicioPrestamos:
    def __init__(self, repo):
        self.repo = repo

    def prestar(self, codigo):
        if codigo not in self.repo.libros:
            return "Ese libro no existe"
        if codigo in self.repo.prestados:
            return "Ese libro ya está prestado"
        self.repo.prestados.add(codigo)
        return f"Prestado: {self.repo.libros[codigo]}"
```

```python


# =============== Capa de presentación ===============
class Consola:
    def __init__(self, servicio):
        self.servicio = servicio

    def ejecutar(self, accion, codigo):
        if accion == "prestar":
            print(self.servicio.prestar(codigo))


consola = Consola(ServicioPrestamos(RepositorioLibros()))
consola.ejecutar("prestar", "L1")
consola.ejecutar("prestar", "L1")
consola.ejecutar("prestar", "L9")
```

RETO Resuélvanlo en el mismo archivo

Agreguen la opción de **devolver** un libro:

(**a**) Un método `devolver(codigo)` en `ServicioPrestamos`. Si el libro no estaba prestado, debe devolver el mensaje `"Ese libro no estaba prestado"`.

(**b**) Un `elif accion == "devolver"` en `Consola` que llame a ese método.

Al final del archivo, devuelvan `L1` y vuelvan a prestarlo.

Pista

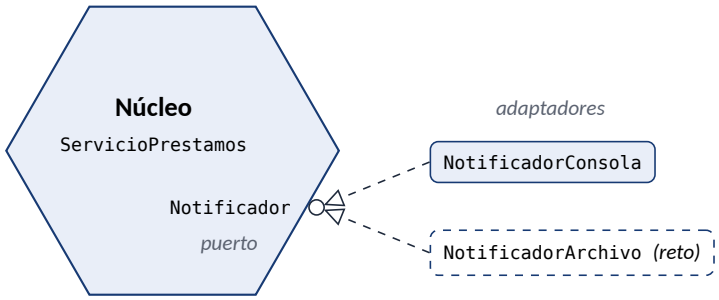
```python
    def devolver(self, codigo):            # en ServicioPrestamos
        if codigo not in self.repo.prestados:
            return "Ese libro no estaba prestado"
        # quitar el código de self.repo.prestados y devolver un mensaje
```

1.2 Hexagonal (puertos y adaptadores)

Entregable: `2_hexagonal.py`

El núcleo no sabe qué tecnología hay afuera

`ServicioPrestamos` tiene que avisarle al estudiante, pero no sabe cómo se envía el aviso: solo conoce el **puerto** `Notificador`. El **adaptador** que se usa hoy es `NotificadorConsola`, que imprime en pantalla.



Código base — `2_hexagonal.py`

```python
"""Arquitectura hexagonal: el núcleo define puertos; afuera, adaptadores."""
from abc import ABC, abstractmethod


# ==================== NÚCLEO ====================
class Notificador(ABC):                    # PUERTO: lo que el núcleo necesita
    @abstractmethod
    def enviar(self, estudiante, mensaje): ...


class ServicioPrestamos:                   # reglas del negocio
    def __init__(self, notificador: Notificador):
        self.notificador = notificador

    def prestar(self, estudiante, titulo):
        self.notificador.enviar(estudiante, f"Te prestamos '{titulo}'")


# ================ ADAPTADORES ================
class NotificadorConsola(Notificador):     # ADAPTADOR: una forma de cumplirlo
    def enviar(self, estudiante, mensaje):
        print(f"[consola] para {estudiante}: {mensaje}")


# ===== Aquí se escoge qué adaptador usar =====
servicio = ServicioPrestamos(NotificadorConsola())
servicio.prestar("ana", "Clean Architecture")
servicio.prestar("luis", "Refactoring")
```

RETO Resuélvanlo en el mismo archivo

Creen un adaptador nuevo, `NotificadorArchivo`, que escriba cada aviso en el archivo `avisos.txt` en vez de imprimirlo. Luego cambien *solo* la línea donde se crea el servicio para que use este adaptador. `ServicioPrestamos` no se toca. Al final, impriman el contenido de `avisos.txt`.

Pista

```
class NotificadorArchivo(Notificador):
    def enviar(self, estudiante, mensaje):
        with open("avisos.txt", "a", encoding="utf-8") as archivo:
            archivo.write(...)   # el aviso y un salto de línea
```

Presentación

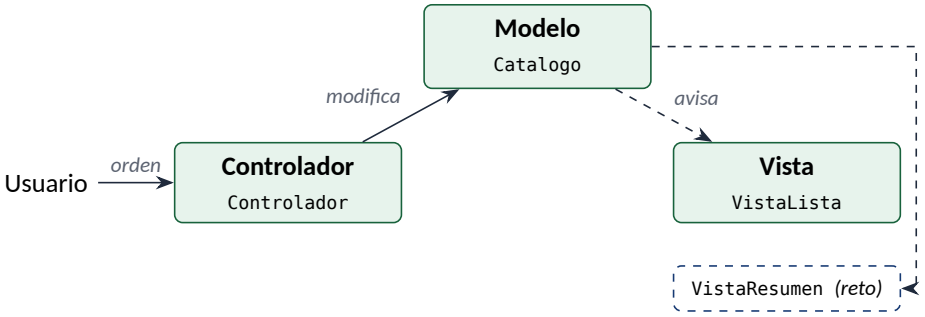
Modelo–Vista–Controlador.

2.1 Modelo–Vista–Controlador (MVC)

Entregable: `3_mvc.py`

El modelo avisa, las vistas muestran

`Catalogo` es el modelo, `VistaLista` es la vista y `Controlador` recibe las órdenes del usuario. Cuando el modelo cambia, llama a `mostrar()` de cada vista que tenga en su lista `vistas`.



Código base — `3_mvc.py`

```python
"""Modelo-Vista-Controlador: catálogo de la biblioteca."""


# ==================== MODELO ====================
class Catalogo:                          # datos + avisa cuando cambia
    def __init__(self):
        self.libros = {"Clean Architecture": True, "Refactoring": True,
                       "Design Patterns": True}
        self.vistas = []

    def prestar(self, titulo):
        self.libros[titulo] = False
        for vista in self.vistas:
            vista.mostrar(self)


# ==================== VISTA ====================
class VistaLista:                        # solo muestra
    def mostrar(self, catalogo):
        for titulo, libre in catalogo.libros.items():
            estado = "disponible" if libre else "prestado"
            print(f"  {titulo}: {estado}")


# ================== CONTROLADOR ==================
```

```python
class Controlador:                      # recibe la orden del usuario
    def __init__(self, catalogo):
        self.catalogo = catalogo

    def orden(self, texto):
        print(f"> {texto}")
        if texto.startswith("prestar "):
            self.catalogo.prestar(texto[len("prestar "):])


catalogo = Catalogo()
catalogo.vistas.append(VistaLista())
controlador = Controlador(catalogo)
controlador.orden("prestar Refactoring")
```

RETO Resuélvanlo en el mismo archivo

Creen una segunda vista, `VistaResumen`, que imprima cuántos libros quedan disponibles (`Disponibles: 2 de 3`). Conéctenla agregándola a `catalogo.vistas`. `Catalogo` y `Controlador` no se modifican.

Pista

```python
class VistaResumen:
    def mostrar(self, catalogo):
        libres = sum(catalogo.libros.values())   # True cuenta como 1
        # imprimir "Disponibles: X de Y"
```

Flujo y comunicación

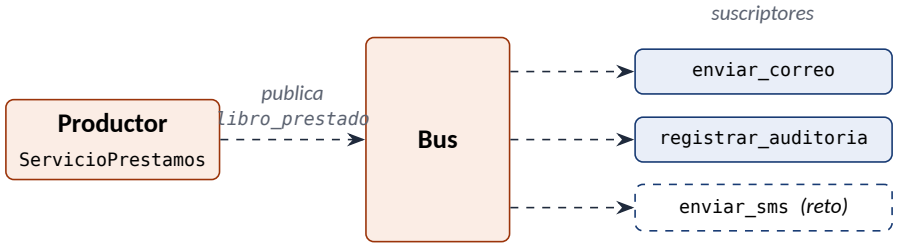
Eventos y tuberías y filtros.

3.1 Orientado a eventos

Entregable: `4_eventos.py`

Publicar un evento y que reaccione quien esté suscrito

`ServicioPrestamos` publica el evento `"libro_prestado"` en el `Bus`. Todas las funciones suscritas a ese evento se ejecutan. El servicio no sabe cuáles son.



Código base — `4_eventos.py`

```python
"""Arquitectura orientada a eventos: publicar y suscribir."""


# ================= BUS DE EVENTOS =================
class Bus:
    def __init__(self):
        self.suscriptores = {}              # evento -> lista de funciones

    def suscribir(self, evento, funcion):
        self.suscriptores.setdefault(evento, []).append(funcion)

    def publicar(self, evento, datos):
        for funcion in self.suscriptores.get(evento, []):
            funcion(datos)


# ================== PRODUCTOR ==================
class ServicioPrestamos:                    # no sabe quién escucha
    def __init__(self, bus):
        self.bus = bus

    def prestar(self, estudiante, titulo):
        print(f"* {estudiante} se lleva '{titulo}'")
        self.bus.publicar("libro_prestado", (estudiante, titulo))


# ================= CONSUMIDORES =================
```

```python
def enviar_correo(datos):
    estudiante, titulo = datos
    print(f"   [correo] {estudiante}, recuerda devolver '{titulo}'")


def registrar_auditoria(datos):
    print(f"   [auditoría] préstamo registrado: {datos}")


bus = Bus()
bus.suscribir("libro_prestado", enviar_correo)
bus.suscribir("libro_prestado", registrar_auditoria)

servicio = ServicioPrestamos(bus)
servicio.prestar("ana", "Clean Architecture")
```

RETO Resuélvanlo en el mismo archivo

Agreguen la función `enviar_sms`, que imprima `[sms] ana: prestaste 'Clean Architecture'`, y suscríbanla al evento `"libro_prestado"`. `ServicioPrestamos` no se toca.

Pista

```python
def enviar_sms(datos):
    estudiante, titulo = datos
    # imprimir el mensaje

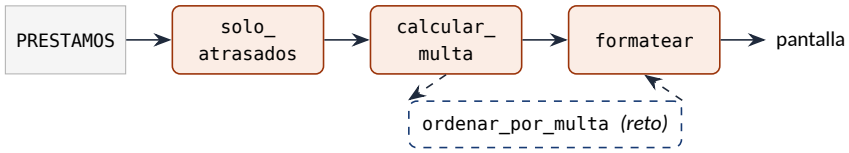
bus.suscribir("libro_prestado", ...)
```

3.2 Tuberías y filtros

Entregable: `5_pipes_filters.py`

Los datos pasan por varios filtros, en orden

El reporte de multas se arma pasando la lista de préstamos por tres filtros: `solo_atrasados`, `calcular_multa` y `formatear`. Cada filtro recibe una lista y devuelve otra, y la función `tuberia` los ejecuta en el orden de la lista.



Código base — `5_pipes_filters.py`

```python
"""Tuberías y filtros: reporte de libros atrasados."""

PRESTAMOS = [
    {"estudiante": "ana", "titulo": "Clean Architecture", "dias": 3},
    {"estudiante": "marta", "titulo": "Design Patterns", "dias": 17},
    {"estudiante": "luis", "titulo": "Refactoring", "dias": 20},
]


# ===== FILTROS: cada uno recibe una lista y devuelve otra =====
def solo_atrasados(prestamos):              # más de 14 días
    return [p for p in prestamos if p["dias"] > 14]


def calcular_multa(prestamos):              # $500 por día de atraso
    return [{**p, "multa": (p["dias"] - 14) * 500} for p in prestamos]


def formatear(prestamos):
    return [f"{p['estudiante']}: {p['titulo']} -> ${p['multa']}"
            for p in prestamos]


# ===== TUBERÍA: pasa los datos por los filtros, en orden =====
def tuberia(datos, filtros):
    for filtro in filtros:
        datos = filtro(datos)
    return datos


for linea in tuberia(PRESTAMOS, [solo_atrasados, calcular_multa, formatear]):
    print(linea)
```

RETO Resuélvanlo en el mismo archivo

Escriban un filtro nuevo, `ordenar_por_multa`, que ordene los préstamos de mayor a menor multa, y pónganlo en la tubería antes de `formatear`. Los otros filtros no se modifican. Luis ($3000) debe quedar de primero.

Pista

```
def ordenar_por_multa(prestamos):
    return sorted(prestamos, key=lambda p: ..., reverse=True)
```